

Yes. And I think this is the point where we should stop talking about “Phase B” and “Phase C” as vague boxes and turn them into an **actual engineering plan**.

Your target architecture should be:



The important thing is that **you do not throw away Phase A**. Your Python implementation becomes the *golden reference* against which both the C++ implementation and FPGA hardware are tested.

Your current report already defines Phase B as C/C++ with fixed-point arithmetic and static escrow allocation, and Phase C as FPGA implementation of the progressive DWT and initial guardrail processing.

1. First: What are we actually building?

Ultimately, I would target the **Terasic DE1-SoC** as your Phase-C platform.

It is a very good fit because it gives you two worlds on one board:



ARM HPS
Cortex-A9

AXI



FPGA Fabric

Cyclone V

DWT IP

Guardrail IP

Buffers

So you can demonstrate a genuinely interesting heterogeneous architecture:

ARM = control/software

FPGA = signal processing acceleration

That is much stronger than simply running your Python code on a laptop.

2. Before anything else: freeze Phase A

This is important.

Do **not** start modifying your existing Python code while developing Phase B.

Create a mental boundary:

text

GOLDEN MODEL

Phase B

C/C++

Phase C

FPGA

Your Python code becomes your **oracle**.

If the FPGA gives a different answer:

“Which implementation is correct?”

You go back to Python.

That makes the project much easier to debug.

3. PHASE B — Embedded C/C++ implementation

Objective

The objective of Phase B is:

Convert the Python reference architecture into deterministic embedded software that can execute within realistic memory, compute and timing constraints.

You're not trying to make it fast yet.

You're trying to make it:

- deterministic,
 - memory bounded,
 - portable,
 - numerically controlled,
 - testable,
 - hardware-independent.
-

4. Where do you implement Phase B?

I recommend:

text

```
event-vault-R/  
|  
|—— eventvault/           ← DO NOT TOUCH  
|   |—— ...               ← Python golden model  
|  
|—— embedded/             ← NEW  
|   |—— include/  
|   |—— src/  
|   |—— tests/  
|   |—— CMakeLists.txt  
|  
|—— verification/         ← NEW  
|   |—— vectors/  
|   |—— python_vs_cpp/  
|   |—— reports/  
|  
|—— experiments/
```

For development, you can initially compile and run the C++ code on your normal PC.

Then move the exact same C++ application onto the DE1-SoC's ARM HPS.

That is important because debugging directly on the FPGA board from day one will be painful.

5. What Python module becomes what?

This is your translation map.

Phase A Python	Phase B C/C++	Phase C
acquisition/synthetic.py	Test-data generator / host-side	Test stimulus
acquisition/calibration.py	calibration.cpp	CPU
codec/dwt.py	dwt.cpp	FPGA
codec/serializer.py	serializer.cpp	CPU
science/source_extraction.py	source_extraction.cpp	CPU initially
science/guardrail.py	guardrail.cpp	FPGA candidate
escrow/buffer.py	escrow_buffer.cpp	CPU memory
escrow/entry.py	escrow_entry.hpp	CPU
escrow/promotion.py	promotion.cpp	CPU
uncertainty/estimator.py	uncertainty.cpp	CPU
uncertainty/retention.py	retention.cpp	CPU
resource/controller.py	resource_controller.cpp	CPU
telemetry/downlink.py	telemetry.cpp	CPU
pipeline.py	pipeline.cpp	CPU orchestrator

This is **not** a one-to-one mandatory mapping. It is the architecture I would recommend.

The Python modules already have this separation, which is precisely why your Phase-A architecture is useful.

6. Phase B, Step 1 — Build the C++ skeleton

Start ridiculously simple.

Create:

text

```
embedded/  
├── include/  
│   ├── dwt.hpp  
│   ├── guardrail.hpp  
│   ├── escrow.hpp  
│   ├── uncertainty.hpp  
│   ├── resource_controller.hpp  
│   └── pipeline.hpp  
├── src/  
│   ├── dwt.cpp  
│   ├── guardrail.cpp  
│   ├── escrow.cpp  
│   ├── uncertainty.cpp  
│   ├── resource_controller.cpp  
│   └── pipeline.cpp  
└── main.cpp
```

Don't worry about FPGA yet.

7. Phase B, Step 2 — Implement the DWT

This is your first major engineering task.

Python currently does something conceptually like:

text

```
Image  
↓  
DWT  
↓  
L0  
H1  
H2  
H3
```

C++ should produce exactly the same conceptual result.

Initially:

Use floating point.

Don't jump immediately into fixed-point.

First prove:

text

Python float

≈

C++ float

Then move to:

text

C++ float

↓

C++ fixed point

This gives you three reference points.

text

Python

↓

C++ floating point

↓

C++ fixed point

8. Phase B, Step 3 – Introduce fixed-point arithmetic

Now you investigate the actual hardware-friendly representation.

For example, you might eventually choose something such as:

text

signed 16-bit

Q8.8

or:

text

signed 32-bit

Q16.16

Do not decide this arbitrarily.

You determine it experimentally.

Take your Phase-A dataset and determine:

```
text

minimum coefficient
maximum coefficient
dynamic range
required precision
accumulator range
```

Then choose the smallest representation that satisfies:

$$|\Delta F| < 0.5\%$$

and:

$$|\Delta x| < 0.1 \text{ pixel}$$

for the tested operating range.

Those are your existing science targets.

This becomes a proper engineering result:

“We selected Qm.n because it provided sufficient numerical accuracy while reducing memory and arithmetic cost.”

9. Phase B, Step 4 — Implement the escrow in static memory

This is another major difference.

Python:

```
text

dynamic objects
lists
dictionaries
```

C++ embedded:

```
text

fixed-size memory pool
```

Something conceptually like:

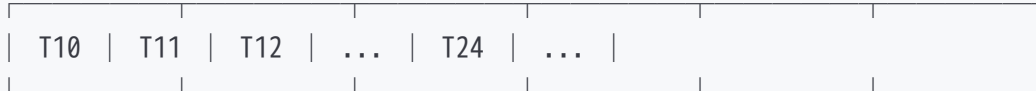
```
text
```

```
MAX_FRAMES = 64
MAX_LAYERS = 3
```

Then:

text

Escrow Memory



Each entry stores metadata plus a reference to its residual data.

The Python escrow behavior already defines the required operations such as push, lock, promotion, expiry and purge.

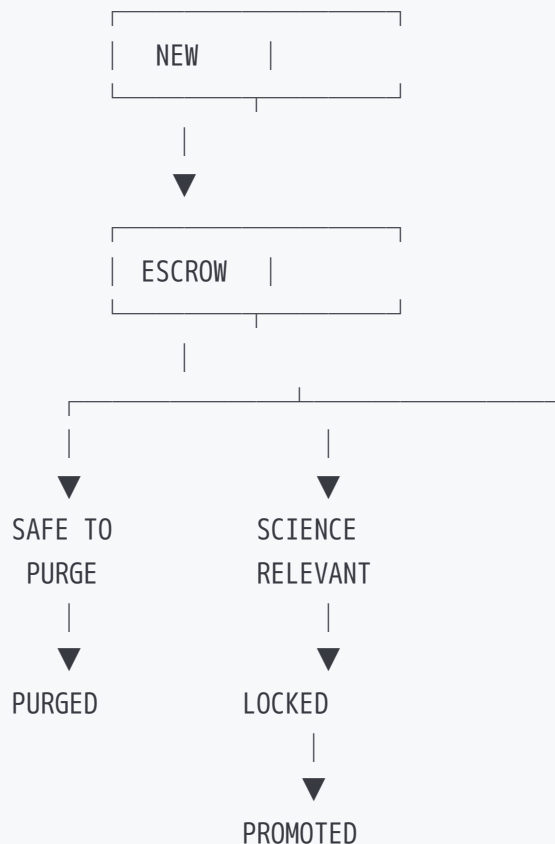
10. Phase B, Step 5 – Implement the state machine

This is worth formalizing.

Every residual layer should have a state.

For example:

text



The C++ implementation should make these transitions explicit.

This is much easier to verify than having retention logic scattered throughout the code.

11. Phase B, Step 6 – Implement the uncertainty system

Port:

```
text

uncertainty/estimator.py
uncertainty/retention.py
```

Initially keep this on the CPU.

You don't need to FPGA-accelerate everything.

The CPU can calculate:

```
text

uncertainty
  ↓
retention priority
  ↓
escrow horizon
```

Your existing Phase-A uncertainty implementation already has entropy and Mahalanobis-based approaches.

12. Phase B, Step 7 – Build the complete C++ pipeline

Eventually:

```
text

main.cpp
  ↓
Acquire
  ↓
Calibrate
  ↓
DWT
  ↓
Source extraction
  ↓
```

```
Guardrail
↓
Uncertainty
↓
Escrow
↓
Resource controller
↓
Promotion
↓
Telemetry
```

At this point you should be able to run:

```
bash

./eventvault
```

and process the exact same Saved Discovery sequence that your Python implementation processes.

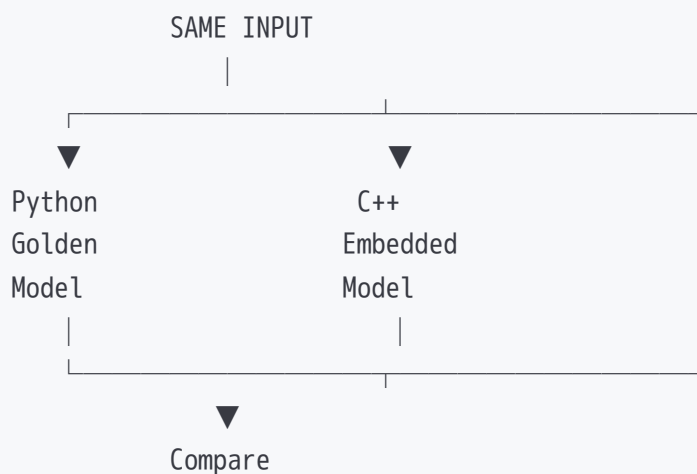
13. This is your first major milestone

You should be able to say:

“We have reproduced the Phase-A algorithmic behavior in embedded C++.”

Then run:

text



Compare:

- DWT coefficients
- reconstructed image
- flux

- centroid
 - guardrail decisions
 - escrow state
 - promotion time
 - recovered frames
-

14. Then put C++ onto the DE1-SoC ARM processor

Now the project gets interesting.

The DE1-SoC has:

text

Cyclone V FPGA
+
ARM Cortex-A9 HPS

You initially ignore the FPGA.

Run EventVault-R entirely on the ARM processor.

So:

text

Camera/Test Data
↓
ARM HPS
↓
C++ EventVault-R
↓
Output

This proves that the **software architecture can run on an embedded target**.

15. Measure it

Now collect real numbers.

For every frame:

text

Processing time
Memory usage

CPU utilisation
Escrow occupancy
Data generated
Data retained
Data transmitted

Your report already identifies execution time, memory usage, power and throughput as important future hardware metrics.

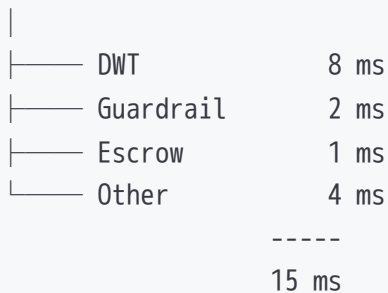
Now you have a baseline.

For example, conceptually:

text

ARM ONLY

Frame



Those numbers are placeholders.

You measure the actual values.

16. Now Phase C starts

The question becomes:

What is slow enough, repetitive enough and deterministic enough to move into hardware?

Your answer should primarily be:

DWT

and potentially:

Flux/centroid/guardrail processing.

Why?

Because they're mathematical streaming operations.

Meanwhile:

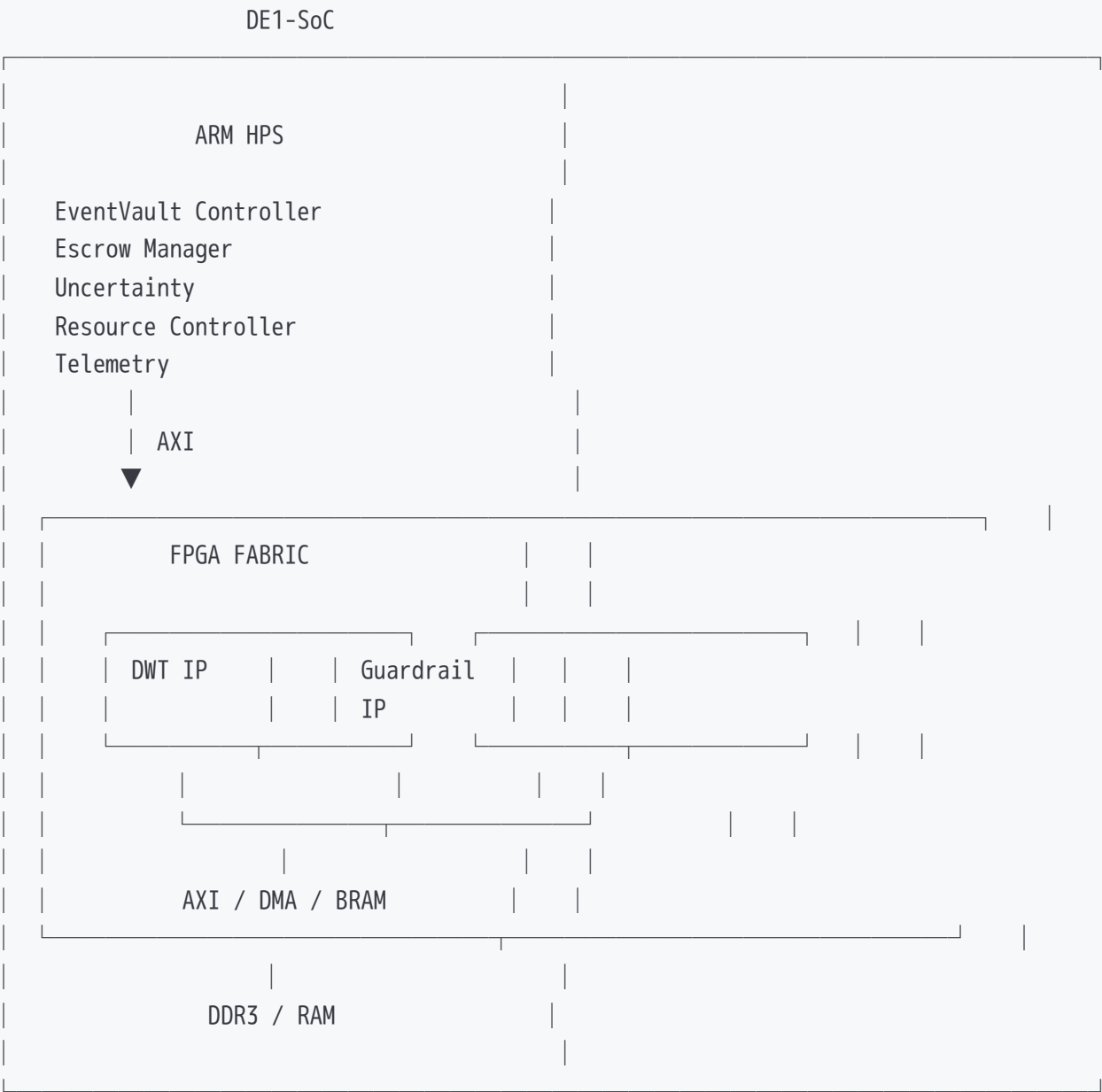
- text
- Escrow
- Promotion
- Uncertainty
- Scheduling
- Telemetry

are much more naturally handled by the ARM processor.

17. FPGA architecture

Your DE1-SoC design becomes:

text



18. FPGA Step 1 – Build the DWT RTL

Start with **one simple block**.

Don't build the entire system.

Create:

```
text

fpga/
├── rtl/
│   └── dwt/
│       ├── dwt_2d.sv
│       ├── dwt_1d.sv
│       ├── lifting_step.sv
│       ├── line_buffer.sv
│       └── ...
└── tb/
    └── dwt_tb.sv
```

I'd recommend **SystemVerilog**, not raw Verilog, for the RTL.

19. The DWT hardware pipeline

The architecture should eventually resemble:



The exact microarchitecture comes after profiling the C++ implementation and choosing the fixed-point representation.

20. FPGA Step 2 – Verify DWT RTL before connecting anything

This is extremely important.

Do:

text

```

Python
  ↓
Golden coefficients
  ↓
SystemVerilog testbench
  ↓
RTL coefficients
  ↓
Compare
  
```

You should have known vectors:

text

```

vectors/
|—— input_001.bin
|—— expected_l0_001.bin
|—— expected_h1_001.bin
|—— expected_h2_001.bin
|—— expected_h3_001.bin
  
```

This is called **golden-vector verification**.

21. FPGA Step 3 – Add the guardrail

Once DWT works:

text

```

DWT
  ↓
coefficients
  ↓
  
```

```
source/measurement processing
↓
guardrail
```

The hardware doesn't need to implement the entire Python `source_extraction.py` immediately.

Start with the deterministic mathematical kernels:

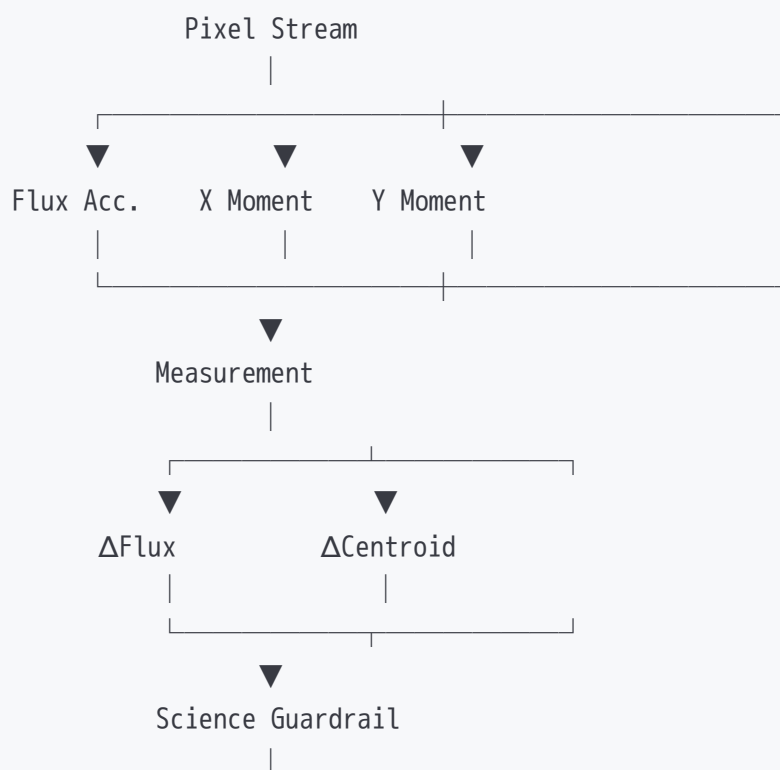
text

```
pixel stream
↓
background subtraction
↓
flux accumulator
↓
x-moment accumulator
↓
y-moment accumulator
↓
flux + centroid
↓
threshold comparator
```

22. Guardrail hardware

Conceptually:

text





LOCK

SAFE

That is a very FPGA-friendly architecture.

23. FPGA Step 4 – Connect the FPGA to the ARM

Now your DE1-SoC finally becomes a heterogeneous accelerator.

ARM sends:

text

Image
configuration
thresholds

to FPGA.

FPGA performs:

text

DWT
Guardrail

and returns:

text

L0
H1/H2/H3
Flux
Centroid
Guardrail status

Then ARM continues:

text

Escrow
Uncertainty
Promotion
Resource control
Telemetry

24. The communication interface

For the first implementation, you can use an AXI-based memory-mapped interface.

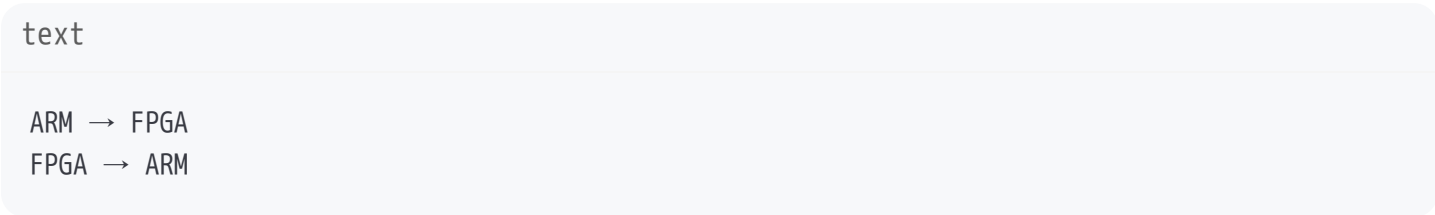
Conceptually:



For bulk image data, you eventually want a DMA path rather than manually pushing every pixel through software.

But **don't start with DMA**.

First get:



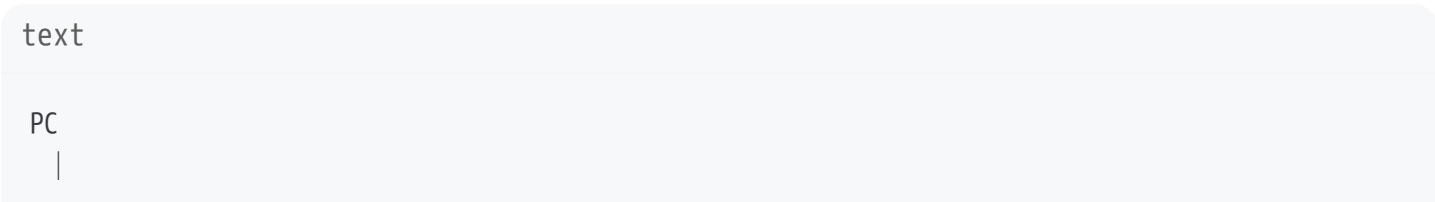
working with a simple interface.

Then optimize.

25. Your final DE1-SoC demo

This is what I would target.

Connect:





You don't actually need a physical astronomical camera.

Your Phase-A synthetic FITS/images can be the sensor stimulus.

That is completely legitimate for a laboratory prototype.

26. The live demonstration

Imagine your professor sitting in front of the board.

You run:

```
bash

./eventvault_demo
```

The board processes:

```
text

T10
T11
T12
...
T25
```

At T10:

text

Transient confidence: LOW
Residuals: ESCROW

At T20:

text

Transient confidence: RISING
Residuals: RETAIN

At T25:

text

TRANSIENT DETECTED

Then:

text

Searching escrow...

T10 PROMOTED
T11 PROMOTED
T12 PROMOTED
...
T24 PROMOTED

And finally:

text

Pre-trigger observations recovered: 100%
Uncontrolled overflow: 0

That's your TRL-4 story.

27. How you prove the FPGA is actually useful

You need a comparison.

ARM-only

text

Input
↓
ARM DWT
↓
ARM Guardrail
↓
ARM EventVault

Measure:

text

Latency
Throughput
CPU utilization
Power

ARM + FPGA

text

Input
↓
FPGA DWT
↓
FPGA Guardrail
↓
ARM EventVault

Measure the same things.

Then produce:

text

	ARM	ARM + FPGA
Latency	X ms	Y ms
Throughput	X fps	Y fps
CPU load	X %	Y %
Power	X W	Y W

Now you can demonstrate an actual hardware acceleration benefit rather than merely saying:

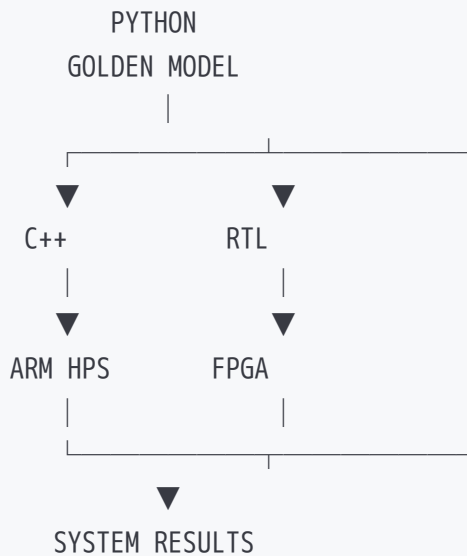
“We put our algorithm on an FPGA.”

28. Your verification hierarchy

This is perhaps the most important part of the entire project.

You should have:

text



And every stage gets tested against the stage above it.

29. The actual development order

Do not try to do everything simultaneously.

I would follow this exact order:

PHASE B

B1 – C++ project skeleton

↓

B2 – Port DWT

↓

B3 – Verify C++ DWT against Python

↓

B4 – Fixed-point DWT

↓

B5 – Verify fixed-point accuracy

↓

B6 – Port Science Guardrail



B7 – Port Escrow



B8 – Port Uncertainty



B9 – Port Resource Controller



B10 – Port complete pipeline



B11 – Run Saved Discovery



B12 – Run Rate-Distortion



B13 – Profile CPU/memory/time



B14 – Deploy C++ onto DE1-SoC ARM



B15 – Profile again

30. Then Phase C

C1 – Freeze C++ interface



C2 – Generate FPGA golden vectors



C3 – RTL DWT



C4 – DWT simulation



C5 – Compare RTL against Python/C++



C6 – Synthesize DWT



C7 – Put DWT onto DE1-SoC



C8 – Add hardware guardrail



C9 – Connect AXI



C10 – Connect ARM ↔ FPGA



C11 – Integrate with C++ EventVault



C12 – Add DMA/streaming optimization



C13 – Hardware-in-the-loop Saved Discovery



C14 – ARM-only vs FPGA comparison



C15 – Final TRL-4 demonstration

31. What software/tools you'll actually need

Phase B

I'd use:

- **C++17**
- GCC / ARM GCC
- CMake
- GoogleTest or Catch2

- Python + NumPy for golden vectors
- Git
- Linux environment

For DE1-SoC:

- ARM cross compiler
- Linux on HPS **or** a suitable bare-metal/RTOS environment

I'd initially choose **Linux on the HPS** because it dramatically simplifies development and debugging.

Once everything works, you can decide whether a more bare-metal/RTOS-oriented implementation is worthwhile.

32. Phase C

For your DE1-SoC FPGA:

- Intel Quartus Prime
- SystemVerilog
- ModelSim/Questa or the simulator available with your Quartus setup
- Platform Designer
- ARM HPS development environment
- AXI/FPGA-to-HPS interconnect
- SignalTap for hardware debugging

And your FPGA project becomes something like:

text

```
eventvault_fpga/
|
|—— rtl/
|   |—— dwt/
|   |—— guardrail/
|   |—— buffers/
|   |—— interfaces/
|
|—— tb/
|   |—— dwt_tb.sv
|   |—— guardrail_tb.sv
|
|—— quartus/
|   |—— eventvault.qpf
|
```

```
└── software/  
    └── hps_controller/
```

33. What I would NOT do

There are several traps I'd avoid.

✗ Don't FPGA-accelerate everything

You don't need:

text

```
FPGA escrow  
FPGA scheduler  
FPGA telemetry  
FPGA uncertainty
```

Initially.

That's unnecessary complexity.

✗ Don't start with DMA

Get simple ARM ↔ FPGA communication working first.

✗ Don't start with fixed-point RTL

First establish:

text

```
Python → C++ float → C++ fixed-point → RTL fixed-point
```

✗ Don't delete the Python implementation

It is your golden reference.

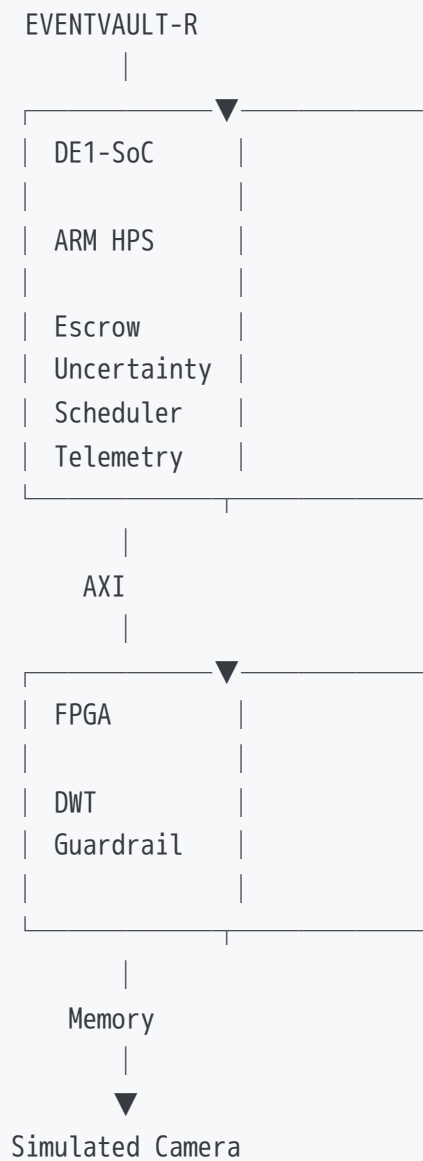
✗ Don't claim TRL-4 just because it runs on DE1-SoC

TRL-4 requires an integrated laboratory demonstration in a representative environment. Your report itself frames the target as an integrated hardware/software demonstrator with measured timing, memory, power and throughput.

34. What your final project will look like

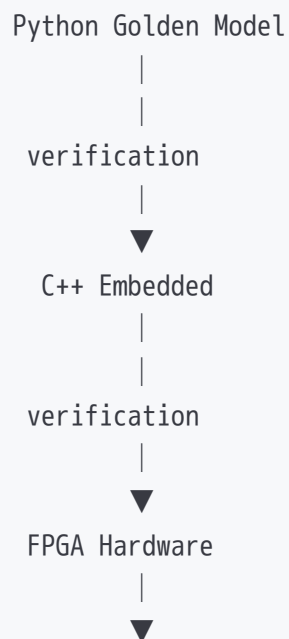
At the end:

text



And the software architecture:

text



35. The most important thing for you right now

Do not start Phase C yet.

Your immediate next milestone should be:

Get Phase-B C++ running on your PC and matching Python.

Specifically, I would make your first sprint:

text

WEEK 1

- ☐ Create embedded/
- ☐ Create CMake project
- ☐ Port DWT
- ☐ Generate Python golden vectors
- ☐ Write C++ DWT
- ☐ Compare Python vs C++
- ☐ Test reconstruction
- ☐ Test L0/H1/H2/H3

Then:

text

WEEK 2

- ☐ Fixed-point DWT
- ☐ Determine Q format
- ☐ Measure numerical error
- ☐ Port flux calculation
- ☐ Port centroid calculation
- ☐ Port Science Guardrail

Then:

text

WEEK 3

- ☐ Port escrow
- ☐ Port promotion
- ☐ Port uncertainty

- ☐ Port resource controller
- ☐ Integrate pipeline

Then:

text

WEEK 4

- ☐ Run Saved Discovery in C++
- ☐ Compare against Python
- ☐ Benchmark
- ☐ Deploy C++ onto DE1-SoC ARM

Only after that do you open Quartus and start writing RTL.

That sequencing will save you an enormous amount of pain.

The beautiful part is that you already have the hardest conceptual work done: the Python reference implementation, the DWT, guardrail, escrow, uncertainty logic, pipeline, experiments and tests. Your next job isn't to invent EventVault-R again. **Your job is to progressively turn the exact thing you already built into deterministic embedded software and then into hardware acceleration.**